

PulseRPM

Remote Patient Monitoring Platform

Complete Build Prompt & Technical Specification

Version 1.0 · June 2026 · Confidential

Document Purpose

This document is a complete build prompt and technical specification for the PulseRPM platform. It covers every component, screen, API endpoint, security mechanism, data flow, and APK build instruction required to reproduce the system from scratch. A developer or AI agent following this document should be able to build a fully functional Remote Patient Monitoring system.

What is PulseRPM?

PulseRPM is a full-stack, HIPAA-aligned Remote Patient Monitoring (RPM) platform consisting of three integrated artifacts: (1) an Express/TypeScript REST API server, (2) a React/Vite provider dashboard, and (3) an Expo React Native Android app. Patients wear smartwatches or use Android Health Connect to automatically sync vitals (heart rate, SpO₂, blood pressure, temperature). The system alerts providers in real-time via email whenever vitals breach configurable thresholds.

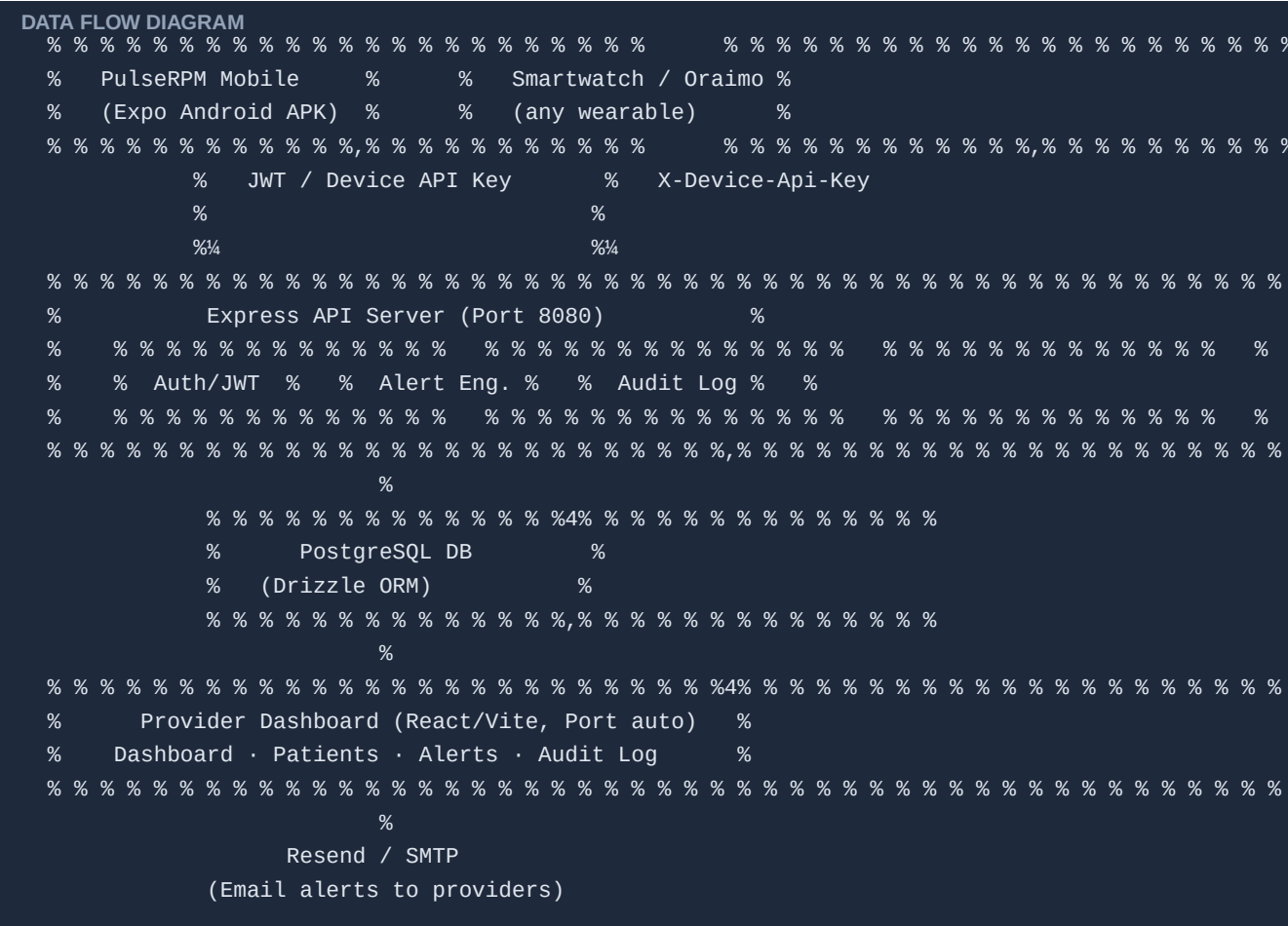
Table of Contents

- §1 System Architecture & Tech Stack
- §2 Repository & Monorepo Structure
- §3 Database Schema (PostgreSQL + Drizzle ORM)
- §4 API Server — Express/TypeScript
- §5 Security Architecture
- §6 Email & Notification System
- §7 RPM Provider Dashboard — React/Vite
- §8 PulseRPM Mobile App — Expo React Native
- §9 Health Connect Integration
- §10 Background Sync Service
- §11 APK Build Instructions (EAS & Android Studio)
- §12 Environment Variables & Secrets
- §13 Demo Credentials & Test Data
- §14 Full API Reference
- §15 Alert Threshold Defaults
- §16 Deployment Notes

§1 System Architecture & Tech Stack

Architecture Overview

PulseRPM uses a three-tier architecture. The Express API server is the single source of truth. The React dashboard and Expo mobile app both communicate with the API over HTTPS. Device data can also be ingested directly from wearables via a public API key endpoint.



Tech Stack

Layer	Technology	Version / Notes
API Server	Express + TypeScript	Node.js 20, ES Modules
ORM	Drizzle ORM	drizzle-kit for migrations
Database	PostgreSQL	Replit managed PostgreSQL
Auth	Custom JWT (HS256)	crypto module, 7-day expiry
Email	Resend API + nodemailer SMTP	HTML templates built-in
Rate Limiting	express-rate-limit	Per-route limits
Security	helmet, cors	CSP, CORS allowlist
Dashboard	React 18 + Vite 5	TypeScript, Tailwind CSS
UI Components	shadcn/ui (Radix)	Accordion, Dialog, Table...
Charts	Recharts	Vitals trend charts
State/Queries	TanStack Query v5	React Query

Mobile

expo-router

Background Sync

expo-background-fetch

15-min interval, dev build only

Storage

@react-native-async-storage

API key, sync logs, pending
queue

Icons

@expo/vector-icons (Feather)

Monorepo

pnpm workspaces

Shared @workspace/db
package

Build

§2 Repository & Monorepo Structure

MONOREPO DIRECTORY STRUCTURE

```
workspace/
% % % packages/
%   % % % db/                # Shared Drizzle schema + client
%       % % % src/schema.ts  # All table definitions
%       % % % src/index.ts   # Exports: db, tables, types
%       % % % drizzle.config.ts
% % % artifacts/
%   % % % api-server/        # Express REST API
%   %   % % % src/
%   %       % % % app.ts      # Express app setup (helmet/cors/rate-limit)
%   %       % % % index.ts    # HTTP server endpoint
%   %       % % % lib/
%   %       %   % % % auth.ts  # JWT + password hashing
%   %       %   % % % alertEngine.ts # Threshold evaluation + alert creation
%   %       %   % % % email.ts  # Resend + SMTP email templates
%   %       %   % % % middlewares/
%   %       %   % % % auth.ts   # requireAuth / requireRole guards
%   %       %   % % % auditLog.ts # Automatic audit trail middleware
%   %       %   % % % rateLimit.ts # auth / api / device limiters
%   %       %   % % % routes/
%   %       %   % % % auth.ts   # /api/auth/*
%   %       %   % % % device.ts # /api/device/* (key + ingest)
%   %       %   % % % vitals.ts # /api/patients/:id/vitals
%   %       %   % % % patients.ts # /api/patients/*
%   %       %   % % % alerts.ts # /api/alerts/*
%   %       %   % % % thresholds.ts # /api/thresholds/*
%   %       %   % % % providers.ts # /api/providers/*
%   %       %   % % % summary.ts # /api/summary
%   %       %   % % % audit.ts # /api/audit
%   % % % rpm-dashboard/      # React/Vite provider dashboard
%   %   % % % src/
%   %       % % % pages/       # login, dashboard, patients, alerts...
%   %       % % % components/ui/ # shadcn/ui components
%   %       % % % lib/auth-context.tsx
%   % % % pulserpm-mobile/    # Expo React Native app
%   %   % % % app/
%   %       % % % _layout.tsx   # Root layout + BackgroundSync import
%   %       % % % pair.tsx      # Device pairing screen
%   %       % % % (tabs)/
%   %       %   % % % _layout.tsx # Tab bar definition
%   %       %   % % % index.tsx   # Monitor tab (vitals dashboard)
%   %       %   % % % log.tsx     # Log Reading (manual entry)
%   %       %   % % % health-connect.tsx # Health Connect sync
%   %       %   % % % settings.tsx # API key / device settings
%   %   % % % services/
%   %       %   % % % HealthConnectService.ts
%   %       %   % % % BackgroundSync.ts
%   %       %   % % % context/AppContext.tsx
%   %       % % % app.json      # Package: com.wgeorge.pulserpm
%   %       % % % eas.json      # EAS build profiles
% % % pnpm-workspace.yaml
% % % package.json
```

§3 Database Schema (PostgreSQL + Drizzle ORM)

All tables live in packages/db/src/schema.ts. The shared @workspace/db package exports the Drizzle client, all table definitions, and TypeScript types for use across the API server.

providers table

Column	Type	Notes
id	serial PRIMARY KEY	
name	text NOT NULL	
email	text NOT NULL UNIQUE	
passwordHash	text NOT NULL	SHA-256 HMAC of password
specialty	text	Optional clinical specialty
role	text DEFAULT "provider"	"provider" "admin"
createdAt	timestamp DEFAULT now()	

patients table

Column	Type	Notes
id	serial PRIMARY KEY	
name	text NOT NULL	
email	text NOT NULL UNIQUE	
passwordHash	text NOT NULL	
dateOfBirth	text	YYYY-MM-DD
gender	text	"male" "female" "other"
conditions	text[] DEFAULT []	e.g. ["Hypertension", "Diabetes"]
deviceType	text DEFAULT "manual"	"manual" "wearable"
deviceApiKey	uuid	Set by generate-key endpoint
providerId	integer REFERENCES providers	Assigned provider
role	text DEFAULT "patient"	
approvalWelcomePending	boolean DEFAULT false	Show welcome banner once
createdAt / updatedAt	timestamp	

pending_patients table

Holds patient self-signup records before provider approval. Deleted when patient is approved.

Column	Type	Notes
id	serial PRIMARY KEY	
name / email	text NOT NULL	
passwordHash	text NOT NULL	
verificationCode	text	6-digit OTP
verificationExpiry	timestamp	Code expires after 15 min

emailVerified

boolean DEFAULT false

vitals table

Column	Type	Notes
id	serial PRIMARY KEY	
patientId	integer REFERENCES patients	
heartRate	integer	BPM
systolicBp	integer	mmHg
diastolicBp	integer	mmHg
spo2	integer	Percentage
temperature	numeric(4,1)	Celsius
caloriesBurned	integer	kcal
source	text DEFAULT "manual"	"manual" "health_connect" "oraimo" "wearable" "healthwear"
recordedAt	timestamp DEFAULT now()	

thresholds table

One row per patient. Created automatically with defaults when first vitals arrive. All fields adjustable per-patient by providers.

Column	Default Value	Description
heartRateMin / Max	50 / 120 BPM	Warning range
heartRateCriticalMin/Max	40 / 150 BPM	Critical range
systolicBpMin / Max	90 / 140 mmHg	Warning range
systolicBpCriticalMin/Max	70 / 180 mmHg	Critical range
diastolicBpMax	90 mmHg	Warning upper
diastolicBpCriticalMax	110 mmHg	Critical upper
spo2Min	95%	Warning lower
spo2CriticalMin	90%	Critical lower
temperatureMin / Max	36.0 / 37.5°C	Warning range
temperatureCriticalMax	39.5°C	Critical upper

alerts table

Column	Type	Notes
id	serial PRIMARY KEY	
patientId	integer REFERENCES patients	
vitalType	text	"heart_rate" "blood_pressure" "spo2" "temperature"
severity	text	"warning" "critical"
message	text	Human-readable description
value	numeric	Actual reading that triggered alert
threshold	numeric	Threshold value that was breached
status	text DEFAULT "active"	"active" "acknowledged" "resolved"
suggestedAction	text	Clinical recommendation
triggeredAt	timestamp DEFAULT now()	
acknowledgedAt / resolvedAt	timestamp	Nullable

audit_logs table

Immutable log of all write operations. Written asynchronously via setImmediate to avoid blocking requests.

Column	Type	Notes
id	serial PRIMARY KEY	
actorId / actorEmail / actorRole	integer / text / text	Who performed the action
action	text	e.g. "vitals.write", "alert.acknowledge"
resourceType / resourceId	text / text	e.g. "patient" / "42"
ipAddress	text	From x-forwarded-for header
userAgent	text	Truncated to 500 chars
outcome	text	"success" "failure" "denied"
details	text	JSON with method, path, status
createdAt	timestamp DEFAULT now()	

§4 API Server — Express/TypeScript

Server Configuration (app.ts)

- Binds to PORT env var (default 8080)
- trust proxy enabled for Replit reverse proxy
- helmet() applied (CSP disabled for cross-origin iframes; crossOriginResourcePolicy: cross-origin)
- CORS allowlist: *.replit.dev, *.replit.app, *.kirk.replit.dev, localhost
- cookieParser() for future cookie-based auth support
- JSON body limit: 1MB
- Rate limiters applied before routing
- Audit middleware applied to all /api routes

Authentication Routes /api/auth/*

POST /api/auth/login

Accepts { email, password }. Checks providers table first, then patients, then pending_patients. Returns JWT token (7-day) + user object. Handles edge cases: email_unverified, pending_approval.

```
POST /api/auth/login
// Request body
{ "email": "provider@hospital.com", "password": "secure_password" }

// Success response
{ "token": "<JWT>", "user": { "id": 1, "email": "...", "name": "...", "role": "provider" } }

// Error states
401 Invalid credentials | 403 Email not verified | 403 Pending approval
```

POST /api/auth/patient-signup

Self-registration flow for patients. Creates pending record, sends 6-digit OTP via Resend. If Resend fails (free tier restriction), returns fallbackCode in response for screen display.

```
POST /api/auth/patient-signup
{ "name": "John Patient", "email": "john@example.com", "password": "secure" }
// !' 201 { message, email, emailSent, fallbackCode? }
```

POST /api/auth/verify-email

Validates 6-digit OTP. Marks pending patient as email-verified. Notifies all providers via email. Expires after 15 min.

POST /api/auth/signup (Provider registration)

Direct provider registration. Creates provider account immediately with JWT. No email verification required.

GET /api/auth/me

Returns current user profile. Requires Authorization: Bearer <token> header.

Device Ingest Routes /api/device/*

POST /api/device/generate-key [Patient JWT required]

Generates a UUID v4 device API key for the authenticated patient. Stores in patients.deviceApiKey.

GET /api/device/status [Public — API key only]

Validates an API key without JWT. Used by mobile app to verify a scanned key before saving. Returns { valid, patientId, patientName }.

POST /api/device/ingest [Public — API key via header]

The primary wearable data ingestion endpoint. Accepts X-Device-API-Key header or ?apiKey query param. Normalizes data from multiple formats (Standard, Apple Health, Fitbit, Google Fit). Runs the alert engine after saving. Returns alertsTriggered count.

```
POST /api/device/ingest — Multi-format payload
// Standard format (used by PulseRPM mobile app)
{
  "heartRate": 72,
  "systolicBp": 120, "diastolicBp": 80,
  "spo2": 98,
  "temperature": 36.8,
  "caloriesBurned": 450,
  "source": "health_connect", // or "oraimo" | "healthwear" | "wearable"
  "recordedAt": "2026-06-20T14:30:00Z"
}

// Apple Health format (auto-normalized)
{ "HeartRate": 72, "OxygenSaturation": 98, "BodyTemperature": 36.8 }

// Google Fit format (auto-normalized)
{ "heartRate": { "bpm": 72 }, "oxygen": { "saturation": 98 } }

// Response
{ "success": true, "vitalsId": 42, "alertsTriggered": 0, "recordedAt": "..." }
```

GET /api/device/expo-dev-url [Public]

Returns the current Expo Go development URL (exp://...) from REPLIT_EXPO_DEV_DOMAIN for QR code generation in the dashboard.

Patient Routes /api/patients/*

Endpoint	Auth	Description
GET /patients	JWT (any role)	List patients. Providers see all; patients see only themselves.
POST /patients	JWT (provider)	Create patient directly (admin flow)
GET /patients/:id	JWT (any role)	Full patient detail with latestVitals, thresholds, risk level
PUT /patients/:id	JWT (provider)	Update patient demographics
GET /patients/pending	JWT (provider)	List pending patients awaiting approval
GET /patients/pending/count	JWT (provider)	Count of pending approvals (for badge)
POST /patients/pending/:id/approve	JWT (provider)	Approve patient: creates patient record, deletes pending, sends email

Vitals Routes /api/patients/:patientId/vitals

Endpoint	Auth	Description
GET /patients/:id/vitals	JWT	List vitals. period=day week month. limit max 500.
POST /patients/:id/vitals	JWT	Ingest vitals + run alert engine. Returns vitals, alerts, riskLevel.
GET /patients/:id/vitals/latest	JWT	Single most-recent reading
POST /vitals/ingest-batch	JWT (provider)	Batch ingest array of { patientId, vitals } objects

§5 Security Architecture

1. JWT Authentication (Custom HS256)

Implemented in packages/api-server/src/lib/auth.ts using Node.js crypto module. No external JWT library dependency.

- Algorithm: HS256 (HMAC-SHA256)
- Secret: JWT_SECRET env var (must be changed in production)
- Token expiry: 7 days (configurable via TOKEN_EXPIRY_SECS constant)
- Payload: { id, email, role, iat, exp }
- Signature verification uses timing-safe comparison via crypto.createHmac
- Expired tokens return null from verifyToken() — treated as 401 Unauthorized

2. Password Hashing

- Algorithm: HMAC-SHA256 keyed with JWT_SECRET
- Function: crypto.createHmac("sha256", JWT_SECRET).update(password).digest("hex")
- No bcrypt/argon2 — hashing is keyed so rainbow tables are ineffective
- Production recommendation: Switch to bcrypt with cost factor 12

3. Role-Based Access Control (RBAC)

- Three roles: "provider", "admin", "patient"
- requireAuth middleware validates JWT and attaches req.user { id, email, role }
- requireRole(...roles) middleware enforces role restrictions per route
- Patients can only access their own records (id check in every patient route)
- Providers see all patients; patients see only themselves
- Device ingest endpoint uses API key (UUID v4) instead of JWT — no session needed

4. Rate Limiting (express-rate-limit)

Limiter	Window	Max Requests	Applied To
authLimiter	15 minutes	10	/api/auth/login, /api/auth/signup
apiLimiter	1 minute	200	All /api/* routes
deviceLimiter	1 minute	60	/api/device/ingest only

5. CORS Policy

- Allowlist pattern matching via regex (not simple string comparison)
 - Allowed: *.replit.dev, *.replit.app, *.kirk.replit.dev, localhost:*
 - All other origins receive CORS error
- Allowed methods: GET, POST, PUT, DELETE, PATCH, OPTIONS
- Allowed headers: Content-Type, Authorization, X-Device-API-Key
 - credentials: true (for future cookie support)

6. Helmet Security Headers

- X-Content-Type-Options: nosniff
- X-Frame-Options: SAMEORIGIN

- X-XSS-Protection: 0
- Strict-Transport-Security: max-age=15552000
- Content-Security-Policy: DISABLED (to allow cross-origin preview iframes)
- Cross-Origin-Resource-Policy: cross-origin (for Replit proxy)

7. Audit Logging

Every write operation (POST, PUT, DELETE, PATCH) is automatically logged to the `audit_logs` table by `auditMiddleware`. Auth routes are excluded from middleware logging (they call `logAuditEvent` manually for finer control).

```
AUDIT LOG STRUCTURE
// Audit log entry fields:
{
  actorId, actorEmail, actorRole, // Who
  action, // e.g. "vitals.write", "patient.update"
  resourceType, resourceId, // What (e.g. "vitals" / "42")
  ipAddress, // From x-forwarded-for (proxy-aware)
  userAgent, // Truncated to 500 chars
  outcome, // "success" | "failure" | "denied"
  details, // JSON: { method, path, statusCode }
  createdAt // Immutable timestamp
}
```

8. Input Validation

- Body parser limits request size to 1MB
- Device API keys must match UUID v4 regex before DB lookup (prevents injection)
- Source field validated against allowlist:

```
["oraimo","healthwear","wearable","manual","apple_health","google_fit","fitbit"]
```

- All numeric vitals cast via `Number()` and checked for `isNaN` before saving
- Vitals rejected if payload contains no recognizable vital signs
- Patient records enforce email uniqueness at both app and DB constraint level

9. Device API Key Security

- Keys are UUID v4 — 122 bits of entropy
- Stored in plaintext in `patients.deviceApiKey` (acceptable for device keys)
- One key per patient — generating a new key invalidates the old one
- Key can be revoked via `DELETE /api/device/key`
- Passed in `X-Device-API-Key` header (not query string preferred)
- UUID format validated via regex before any DB query

§6 Email & Notification System

Email Transport Priority

The email module tries transports in this order: Resend API !' SMTP !' console fallback.

1. If RESEND_API_KEY env var is set: calls <https://api.resend.com/emails> via fetch
2. If SMTP_HOST + SMTP_USER + SMTP_PASS are set: uses nodemailer SMTP transport
3. If neither is configured: logs email content to console (development fallback)

Email Templates (all HTML)

1. Patient Verification Email

Sent during patient self-signup. Contains 6-digit OTP in large styled code block. Expires in 15 minutes. If Resend delivery fails, fallbackCode is included in the API response for on-screen display.

2. Clinical Alert Email

Sent to the patient's assigned provider (or first provider) when vitals trigger an alert. Includes:

- Severity badge (CRITICAL in red / WARNING in amber)
- Patient profile: name, age, gender, known conditions
- Alert table: severity, vital type, reading, recommended clinical action
- Timestamp (UTC)
- Subject line: "[CRITICAL] Patient Alert — Eleanor Thompson"

3. Patient Approved Email

Sent to patient when a provider approves their account. Contains: welcome message, login URL button, list of what they can now do (view vitals, connect wearable, receive alerts).

4. New Patient Pending Email

Sent to all providers when a new patient verifies their email. Contains: patient name, email, pending status badge, "Review in Dashboard" button linking to /patients page.

Resend Configuration

```
RESEND ENV VARS
RESEND_API_KEY=re_xxxx           # Required – get from resend.com
RESEND_FROM_EMAIL=onboarding@resend.dev # Use verified domain in production
RESEND_FROM_NAME=PulseRPM

# Note: Resend free tier only delivers to verified email addresses.
# In development, email may only reach the account owner's email.
# The API will return fallbackCode in the response if delivery fails.
```

§7 RPM Provider Dashboard — React/Vite

Overview

A single-page application built with React 18 + Vite 5 + TypeScript. Uses shadcn/ui (Radix UI primitives + Tailwind CSS) for all UI components. TanStack Query (React Query v5) manages all server state with automatic refetch. Protected by JWT stored in localStorage.

Authentication (auth-context.tsx)

- AuthContext provides: user, token, login(), logout(), loading state
- JWT stored in localStorage key "rpm_token"
- On mount: fetches GET /api/auth/me with stored token to validate session
- All API calls include Authorization: Bearer <token> header
- Logout clears localStorage and redirects to /login

Pages & Screens

/login — Login Page

- Email + password form
- Shows different error messages for email_unverified and pending_approval states
- On success: stores JWT, redirects based on role (provider !' /dashboard, patient !' /patients)
- Patient self-registration tab with OTP verification flow
- Resend verification code button with 60-second cooldown

/dashboard — Provider Dashboard

- Summary cards: total patients, active alerts, critical alerts, avg heart rate
- Recent alerts panel with severity color coding
- Patient list with risk level indicators (green/amber/red)
- Pending patient approval badge with count in navigation
- Auto-refreshes every 30 seconds

/patients — Patient List

- Searchable, filterable table with risk level filter
- Each row: name, age, conditions, risk badge, alert count, last sync time, latest vitals
- Pending approval queue: shows name, email, approve button with optional form for conditions/DOB
- Add Patient button (direct provider creation)
- Risk levels: normal (green), warning (amber), critical (red)

/patients/:id — Patient Detail

- Vital trend charts (Recharts line charts): heart rate, SpO₂, blood pressure, temperature
- Period selector: 24h / 7d / 30d
- Latest vitals cards with color-coded indicators vs thresholds
- Active alerts section with acknowledge/resolve actions
- Threshold editor: per-patient adjustable warning and critical ranges for all vitals
- Device info: device type, API key display, QR code for mobile app pairing

/alerts — Alert Management

- Full list of all alerts across all patients
- Filter by severity (warning / critical) and status (active / acknowledged / resolved)
- One-click acknowledge and resolve actions
- Alert details: patient name, vital type, value, threshold, suggested clinical action, timestamp

/audit-log — Audit Trail

- Paginated table of all write events
- Columns: timestamp, actor (email + role), action, resource, outcome, IP address

- Provider-only access (403 for patients)
- Non-editable — immutable audit trail

/sync — Mobile App Sync

- Shows QR code for scanning in Expo Go (uses GET /api/device/expo-dev-url)
- Device API key management: generate, display, revoke
- Instructions for pairing the mobile app

§8 PulseRPM Mobile App — Expo React Native

App Configuration

```
app.json — key configuration
// app.json key settings
{
  "name": "PulseRPM Mobile",
  "slug": "pulserpm-mobile",
  "scheme": "pulserpm-mobile",
  "android": {
    "package": "com.wgeorge.pulserpm",
    "versionCode": 1,
    "permissions": [
      "android.permission.health.READ_HEART_RATE",
      "android.permission.health.READ_OXYGEN_SATURATION",
      "android.permission.health.READ_BLOOD_PRESSURE",
      "android.permission.health.READ_BODY_TEMPERATURE"
    ]
  },
  "plugins": ["expo-router", "expo-font", "react-native-health-connect"],
  "experiments": { "typedRoutes": true, "reactCompiler": true }
}
```

Navigation Structure (expo-router)

```
NAVIGATION STRUCTURE
app/
% % % _layout.tsx      # Root: AppProvider + BackgroundSync import + Stack
% % % pair.tsx        # Pairing screen (API key entry / QR scan)
% % % (tabs)/         # Tab group
  % % % _layout.tsx    # Tabs: Monitor | Log Reading | Health Connect | Settings
  % % % index.tsx      # Monitor — vitals overview + sync status
  % % % log.tsx        # Log Reading — manual vital entry form
  % % % health-connect.tsx # Health Connect sync + permissions
  % % % settings.tsx   # API key management + device info
```

AppContext (context/AppContext.tsx)

Global React context providing all sync functionality to every screen. Wraps the entire app via AppProvider in `_layout.tsx`.

State managed:

- `apiKey` — UUID device key stored in AsyncStorage
- `paired` — boolean (true when `apiKey` is set)
- `syncStatus` — "idle" | "syncing" | "success" | "error"
- `syncLogs` — last 50 sync attempts with status, reading, alerts triggered
- `totalSynced` — cumulative count
- `pendingCount` — offline readings waiting to retry
- `lastSyncTime` / `lastHCSyncTime` — timestamps for display

Key functions:

- `setApiKey(key)` — saves to AsyncStorage, triggers background sync registration
- `clearApiKey()` — wipes all stored data and state
- `syncReading(vital)` — POSTs to `/api/device/ingest` with X-Device-API-Key, logs result

- `retryPending()` — retries all queued failed readings
- `syncFromHealthConnect()` — reads last 30 min of HC data and syncs if any new readings

Offline support:

- Failed readings are queued in `AsyncStorage` as "pending"
- On foreground wake (`AppState` "active"), `retryPending()` is called automatically
- Prevents data loss when phone has no connectivity

Screens

Monitor Tab (`index.tsx`)

- Sync status card: shows current status with animated indicator
- Latest vitals display: heart rate, SpO₂, blood pressure, temperature with source badge
- Total synced counter and last sync timestamp
- Pending readings indicator with retry button
- Recent sync log (last 50 entries) with success/fail indicators

Log Reading Tab (`log.tsx`)

- Manual vital entry form: heart rate, SpO₂, systolic/diastolic BP, temperature
- Source selector: Manual / Oraimo / Healthwear / Wearable
- Haptic feedback on submit (medium impact)
- Sends directly to API via `syncReading()` in `AppContext`
- Validation: at least one vital required before submit

Health Connect Tab (`health-connect.tsx`)

- In native APK: shows status banner, latest 24h vitals from HC, permission grant button, manual sync button, background sync status
- In Expo Go: shows "Native Build Required" screen with EAS + Android Studio build instructions
- Permission types requested: `HeartRate`, `OxygenSaturation`, `BloodPressure`, `BodyTemperature`
- Sync reads last 24h of HC data and posts to API
- Background sync status indicator (active / pending dev build)

Settings Tab (`settings.tsx`)

- Display current API key (masked)
- Generate new key (with confirmation dialog)
- Revoke key button (with confirmation)
- Server URL display
- App version info

Pair Screen (`pair.tsx`)

- Entry screen shown when no API key is set
- QR code scanner using `expo-camera`
- Manual key entry with paste support
- Validates key via `GET /api/device/status` before saving
- Shows patient name on successful validation

§9 Health Connect Integration

Why Health Connect Cannot Run in Expo Go

Important Limitation

react-native-health-connect is a native Android module.

Native modules must be compiled into the APK — they cannot be injected at runtime.

Expo Go is a generic sandbox that only supports built-in Expo SDK modules.

Solution: Build a standalone APK via EAS Build (see §11).

HealthConnectService.ts — API

getHCStatus() !' HCStatus

- "android_only" — not on Android device
- "unavailable" — native module failed to load (Expo Go)
- "not_installed" — Health Connect app needs update (getSdkStatus returns SDK_UNAVAILABLE_PROVIDER_UPDATE_REQUIRED)
- "not_supported" — Android version < 8.0
- "ready" — Health Connect initialized and ready

getHCPpermissions() !' HCPpermissions

Calls hc.getGrantedPermissions(). Returns { heartRate, spo2, bloodPressure, temperature } booleans.

requestHCPpermissions() !' HCPpermissions

Calls hc.requestPermission([HeartRate, OxygenSaturation, BloodPressure, BodyTemperature]). Returns same shape. Opens the Android Health Connect permission dialog.

readLatestVitals(hoursBack = 24) !' HCVitalReading

Reads all 4 record types for the last N hours. Returns { heartRate?, spo2?, systolicBp?, diastolicBp?, temperature? }. Each field populated from the most recent record found.

```
readLatestVitals — internal logic
// Internal flow of readLatestVitals
const hc = await ensureInit();
const timeRangeFilter = {
  operator: "between",
  startTime: from.toISOString(),
  endTime: now.toISOString(),
};

// HeartRate: uses samples[].beatsPerMinute from latest record
// OxygenSaturation: percentage field (auto-scaled if <= 1 !' multiply by 100)
// BloodPressure: systolic.inMillimetersOfMercury + diastolic.inMillimetersOfMercury
// BodyTemperature: temperature.inCelsius
```

Lazy Importing (Critical)

HealthConnectService and BackgroundSync are imported lazily via import() to prevent crashes in Expo Go:

```
LAZY IMPORT PATTERN
// In AppContext.tsx — safe dynamic import
const { getHCStatus, readLatestVitals, hasAnyReading } = await import(
  "@services/HealthConnectService"
);
```


Android Manifest Permissions (app.json)

REQUIRED ANDROID PERMISSIONS

```
"android": {  
  "permissions": [  
    "android.permission.health.READ_HEART_RATE",  
    "android.permission.health.READ_OXYGEN_SATURATION",  
    "android.permission.health.READ_BLOOD_PRESSURE",  
    "android.permission.health.READ_BODY_TEMPERATURE"  
  ]  
}
```

§10 Background Sync Service

Architecture

Background sync uses expo-background-fetch + expo-task-manager. The task runs every 15 minutes when the app is in the background or closed. It reads Health Connect data and posts to the API.

Critical Implementation Rule

TaskManager.defineTask() MUST be called at module scope — before any React renders. This means BackgroundSync.ts must be imported at the TOP of app/_layout.tsx. Importing it inside a component or useEffect is too late — the task will not register. Pattern: import "@services/BackgroundSync"; at top of _layout.tsx

Task Flow

```
BACKGROUND TASK DEFINITION
// HC_SYNC_TASK = "pulserpm-hc-background-sync"
TaskManager.defineTask(HC_SYNC_TASK, async () => {
  // 1. Check platform (Android only)
  // 2. Load API key from AsyncStorage
  // 3. Check HC status (must be "ready")
  // 4. Check granted permissions
  // 5. readLatestVitals(0.5) !• last 30 minutes only
  // 6. POST to /api/device/ingest with X-Device-Api-Key
  // 7. Update STORAGE_KEY_LAST_HC_SYNC timestamp
  // Returns: NewData | NoData | Failed
});
```

Registration

```
TASK REGISTRATION
await BackgroundFetch.registerTaskAsync(HC_SYNC_TASK, {
  minimumInterval: 15 * 60, // 15 minutes
  stopOnTerminate: false,   // continues after app is killed
  startOnBoot: true,        // auto-starts after device reboot
});
```

Expo Go Behaviour

BackgroundFetch.registerTaskAsync() throws in Expo Go. The catch block silently swallows the error and returns false. The health-connect.tsx screen shows "Background sync pending (dev build required)" when bgSyncRegistered === false.

On Foreground Wake

AppContext listens to RNAppState changes. When the app comes to foreground from background/inactive, it runs:

- retryPending() — flushes the offline queue
- syncFromHealthConnect() — reads last 30 min of HC data

§11 APK Build Instructions

EAS Build Profiles (eas.json)

```
eas.json
{
  "cli": { "version": ">= 12.0.0" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "android": { "buildType": "apk" },
      "distribution": "internal"
    },
    "production": {
      "android": { "buildType": "app-bundle" }
    }
  }
}
```

Option A: EAS Cloud Build (Recommended — No Android Studio Required)

Build time

EAS cloud builds take approximately 10 minutes.

Free tier: 30 build minutes/month. Enough for several preview APKs.

After build: scan QR code from expo.dev to install directly on device.

1. Install EAS CLI globally: `npm install -g eas-cli`
2. Log in to Expo account (free): `eas login`
3. Navigate to mobile app directory: `cd artifacts/pulserpm-mobile`
4. Set `EXPO_PUBLIC_DOMAIN` env var to your API server domain (see §12)
5. Run preview build (produces installable APK): `eas build --profile preview --platform android`
6. When build completes, scan the QR code from `expo.dev/builds` to install on Android device
7. Open PulseRPM app !' Health Connect tab !' Grant permissions !' Sync Now

Option B: Development Build (for developers with Expo account)

1. `cd artifacts/pulserpm-mobile`
2. `eas build --profile development --platform android`
3. Install the development client APK on device
4. Run: `npx expo start --dev-client`
5. Scan QR code to open in development client (supports hot reload + HC)

Option C: Android Studio Local Build

1. Prerequisites: Android Studio + Android SDK API 33+, Java 17
2. `cd artifacts/pulserpm-mobile`
3. Set `EXPO_PUBLIC_DOMAIN` in `.env.local`: `EXPO_PUBLIC_DOMAIN=your-api-domain.replit.dev`
4. Run prebuild: `npx expo prebuild --platform android --clean`
5. Open `android/` folder in Android Studio
6. Click "Run app" on connected device or AVD (API 33+)
7. For release APK: Build > Generate Signed Bundle/APK > APK

Health Connect Requirements

- Minimum Android version: 8.0 (API 26)
- Health Connect app must be installed (bundled in Android 14+; free on Play Store for older versions)
- Supported wearables: any that sync to Health Connect (Galaxy Watch, Pixel Watch, Fitbit, Garmin, etc.)
- Oraimo watches sync to the Oraimo app, which writes to Health Connect

After Installing APK

1. Open PulseRPM mobile app
2. Go to Health Connect tab
3. Tap "Grant Health Connect Permissions"
4. Allow: Heart Rate, Blood Oxygen, Blood Pressure, Body Temperature
5. Return to Health Connect tab — status shows "Health Connect Active"
6. Tap "Sync Now" to push latest 24h readings immediately
7. Background sync (every 15 min) activates automatically

Troubleshooting APK Issues

Issue	Solution
Health Connect shows "Not Available"	Must use preview/development build — not Expo Go
Background sync not running	Check Android battery optimization — exclude PulseRPM from battery saver
No permissions dialog appears	Health Connect app may need update from Play Store
Sync fails after pairing	Check EXPO_PUBLIC_DOMAIN matches your running API server URL
API key invalid	Generate new key from dashboard Settings !' Sync page

§12 Environment Variables & Secrets

API Server (.env)

```
API SERVER ENVIRONMENT VARIABLES
# Required
DATABASE_URL=postgresql://user:pass@host:5432/db
JWT_SECRET=your-very-long-random-secret-min-32-chars

# Email (at least one required for email delivery)
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
RESEND_FROM_EMAIL=alerts@yourdomain.com
RESEND_FROM_NAME=PulseRPM

# Optional SMTP fallback
SMTP_HOST=smtp.gmail.com
SMTP_USER=your@gmail.com
SMTP_PASS=your-app-password
SMTP_PORT=587

# Optional
PORT=8080 # Default: 8080
DASHBOARD_URL=https://your-app.replit.app # Used in approval email links
```

Mobile App (.env or app.config.js)

```
MOBILE APP ENVIRONMENT VARIABLES
# Required — must point to running API server
EXPO_PUBLIC_DOMAIN=your-api-server-domain.replit.dev

# The mobile app constructs:
# INGEST_URL = https://${EXPO_PUBLIC_DOMAIN}/api/device/ingest
```

Security Notes for Production

- JWT_SECRET must be at least 32 random characters — generate with: `openssl rand -hex 32`
- Never commit .env files to version control
- Resend free tier: emails only deliver to verified addresses. Use a paid plan or verified domain for production.
- SMTP_PASS for Gmail must be an App Password (not account password) — enable 2FA first
- In production, run behind HTTPS only — Replit deployments handle TLS automatically
- Consider rotating JWT_SECRET periodically (will invalidate all existing sessions)

§13 Demo Credentials & Test Data

Default Accounts

Account Type	Email	Password	Role
Provider	sarah.mitchell@rpmhospital.com	password123	provider
Patient	eleanor.thompson@email.com	patient123	patient

These accounts are seeded during database initialization. The provider account has full access to the dashboard. The patient account can view their own vitals and sync from the mobile app.

Seeding the Database

The API server seeds demo data automatically on first start if the database is empty. This includes the two accounts above, sample vitals readings, and pre-configured alert thresholds for the demo patient.

Test Vitals (device/ ingest)

TEST COMMANDS

Normal reading

```
curl -X POST https://YOUR_API_DOMAIN/api/device/ingest \
  -H "X-Device-API-Key: <patient-device-api-key>" \
  -H "Content-Type: application/json" \
  -d '{"heartRate":72,"systolicBp":120,"diastolicBp":80,"spo2":98,"temperature":36.8,"source":"oraimo"}'
```

Critical reading (triggers alert email)

```
curl -X POST https://YOUR_API_DOMAIN/api/device/ingest \
  -H "X-Device-API-Key: <patient-device-api-key>" \
  -H "Content-Type: application/json" \
  -d '{"heartRate":155,"systolicBp":185,"spo2":88,"temperature":39.8,"source":"wearable"}'
```

§14 Full API Reference

Base URL

All endpoints are prefixed with /api. Example: <https://your-api.replit.dev/api/auth/login>

Authentication Header

HEADERS

Authorization: Bearer <JWT_TOKEN>

X-Device-API-Key: <UUID_V4_DEVICE_KEY> # For device/ingest only

All Endpoints

Method	Path	Auth	Description
POST	/auth/login	None	Login (provider or patient)
POST	/auth/signup	None	Provider registration
POST	/auth/patient-signup	None	Patient self-registration
POST	/auth/verify-email	None	Submit 6-digit OTP
POST	/auth/resend-code	None	Re-send OTP email
GET	/auth/me	JWT	Get current user
POST	/auth/dismiss-welcome	JWT (patient)	Dismiss welcome banner
POST	/device/generate-key	JWT (patient)	Generate device API key
GET	/device/key	JWT (patient)	Get current device API key
DELETE	/device/key	JWT (patient)	Revoke device API key
GET	/device/status	Device key	Validate API key (public)
POST	/device/ingest	Device key	Ingest vitals from wearable
GET	/device/expo-dev-url	None	Get Expo Go dev URL
GET	/patients	JWT	List patients
POST	/patients	JWT (provider)	Create patient (admin)
GET	/patients/pending	JWT (provider)	List pending approvals
GET	/patients/pending/count	JWT (provider)	Count pending approvals
POST	/patients/pending/:id/approve	JWT (provider)	Approve patient
GET	/patients/:id	JWT	Patient detail
PUT	/patients/:id	JWT (provider)	Update patient
GET	/patients/:id/vitals	JWT	Vitals list (period, limit)
POST	/patients/:id/vitals	JWT	Add vitals reading
GET	/patients/:id/vitals/latest	JWT	Most recent vitals
POST	/vitals/ingest-batch	JWT (provider)	Batch vitals ingest
GET	/alerts	JWT	List alerts
POST	/alerts/:id/acknowledge	JWT	Acknowledge alert
POST	/alerts/:id/resolve	JWT	Resolve alert
GET	/thresholds/:patientId	JWT	Get patient thresholds
PUT	/thresholds/:patientId	JWT (provider)	Update thresholds

GET

/summary

JWT (provider)

GET

/audit

JWT (provider)

GET

/health

None

§15 Alert Threshold Defaults & Engine

Default Threshold Values

Vital	Warning Low	Warning High	Critical Low	Critical High
Heart Rate	50 BPM	120 BPM	40 BPM	150 BPM
Systolic BP	90 mmHg	140 mmHg	70 mmHg	180 mmHg
Diastolic BP	—	90 mmHg	—	110 mmHg
SpO ₂	95%	—	90%	—
Temperature	36.0°C	37.5°C	—	39.5°C

Alert Engine Logic (evaluateVitals)

Runs after every vital submission (both from mobile app and direct ingest). Checks each vital independently against both warning and critical thresholds. A reading can trigger multiple alerts simultaneously (e.g., both heart rate and blood pressure critical).

```
ALERT ENGINE PSEUDOCODE
// evaluateVitals returns AlertCandidate[]
// Each candidate becomes a database alert row + triggers email

// Heart Rate example
if (hr <= criticalMin || hr >= criticalMax) {' severity: "critical"
else if (hr < warningMin || hr > warningMax) {' severity: "warning"

// SpO2 (lower bound only)
if (spo2 <= criticalMin) {' severity: "critical"
else if (spo2 < warningMin) {' severity: "warning"

// Each alert has a suggestedAction for the provider, e.g.:
// "Contact patient immediately. Consider emergency response."
// "Review blood pressure medication. Advise patient to rest."
```

Alert Processing Flow

1. Vital saved to vitals table
2. evaluateVitals() checks all present fields against thresholds
3. processAndSaveAlerts() inserts alert rows into alerts table
4. setImmediate() fires email notification to provider asynchronously (non-blocking)
5. API responds immediately — email delivery does not delay the response
6. Provider sees new alert badge in dashboard (auto-refreshes every 30s)

Alert States

Status	Description	Transition
"active"	Newly triggered, requires attention	Acknowledge or Resolve
"acknowledged"	Provider has seen it, monitoring	Resolve
"resolved"	Issue resolved, no further action	Terminal state

§16 Deployment Notes

Replit Deployment

All three artifacts (API server, dashboard, mobile dev server) run as separate Replit workflows. Deploying publishes them under *.replit.app domains with automatic TLS.

Workflow Commands

Artifact	pnpm Command	Port
API Server	pnpm --filter @workspace/api-server run dev	8080 (PORT env)
RPM Dashboard	pnpm --filter @workspace/rpm-dashboard run dev	PORT env
Expo Mobile	pnpm --filter @workspace/pulserpm-mobile run dev	Expo default

Production Checklist

- & Set JWT_SECRET to a long random value (openssl rand -hex 32)
- & Set RESEND_API_KEY and verify sender domain with Resend
- & Set DATABASE_URL to production PostgreSQL instance
- & Set DASHBOARD_URL to published dashboard URL (used in approval emails)
- & Set EXPO_PUBLIC_DOMAIN to production API server URL in mobile app build
- & Run database migrations: pnpm --filter @workspace/db run db:push
- & Change default demo account

passwords

- & Build

production APK:

eas build --profile

production --

platform android

- & Submit to

Play Store: eas

submit --profile

production --

platform android

- & Enable

Android battery

optimization

exclusion

instructions in

app onboarding

Monitoring & Maintenance

- Audit log:

review weekly for

suspicious

activity patterns

- Alert

volume: high

alert counts may

indicate threshold

misconfiguration

- Device API

key rotation:

recommend

patients rotate

keys every 90

days

- Database

backups: Replit

PostgreSQL

includes

automatic

backups

- Token

expiry: JWT

expires after 7

days; users re-

login

automatically

